



深入理解 Flexbox

彻底搞懂 flex-grow、flex-shrink
、 flex-basis

目录

1. 深入理解 Flexbox
2. 目录
3. Flexbox 布局困惑
4. 核心概念：三个关键属性
5. Flexbox 的空间分配三步法
6. 扩张与收缩的计算公式
7. 场景一：空间充足 (flex-grow)
8. 场景二：空间不足 (flex-shrink)
9. 总结

内容概览

探索 Flexbox 的核心分配机制。

1. **理解核心**：明确 `flex-grow` , `flex-shrink` , `flex-basis` 三个关键属性。
2. **掌握流程**：学习空间分配的“三步法”决策逻辑。
3. **实战演练**
4. **总结归纳**：回顾核心思想，让 Flexbox 布局不再神秘。

Flexbox 布局困惑

你是否也曾感到困惑：几个子元素在容器中，有的被拉伸，有的被压缩，但最终尺寸似乎难以预测？

- 为什么有些元素变宽了，有些却没有？
- 空间不足时，元素是按什么比例缩小的？
- `flex: 1` 到底意味着什么？

其实，这背后遵循着一套清晰、严谨的数学分配规则。

核心概念：三个关键属性

三个属性的职责。

flex-basis

基准宽度：空间分配的计算起点，元素的“理想”尺寸。

flex-grow

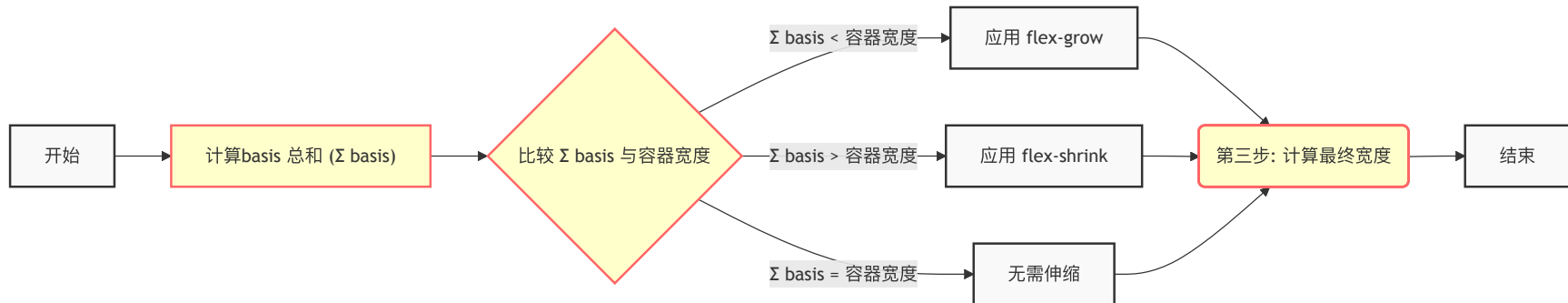
扩张因子：当容器有剩余空间时，决定如何“瓜分”这些空间。

flex-shrink

收缩因子：当容器空间不足时，决定如何“分摊”超出的部分。

Flexbox 的空间分配三步法

整个分配过程，无论扩张还是收缩，都遵循着一个清晰的逻辑。



扩张与收缩的计算公式

根据空间是否充足，我们进入第三步，使用不同的公式计算最终宽度。

扩张公式 (Grow)

当存在剩余空间时，按 `grow` 值比例分配。

$$\text{最终宽度} = \text{basis} + (\text{剩余空间} \times \frac{\text{grow}}{\Sigma \text{grow}})$$

收缩公式 (Shrink)

当空间不足时，按 `basis * shrink` 权重分摊。

$$\text{最终宽度} = \text{basis} - (\text{超出空间} \times \frac{\text{basis} \times \text{shrink}}{\Sigma (\text{basis} \times \text{shrink})})$$

场景一：空间充足 (flex-grow)

设定：

- 容器宽度: 600px
- 子项 A: flex: 1 1 0;
- 子项 B: flex: 2 1 auto; (内容宽度 200px)
- 子项 C: flex: 1 1 100px;

```
// 4. 计算最终宽度
// 最终宽度 = flex-basis + 分配到的空间
const finalWidthA = 0 + 75; // 75px
const finalWidthB = 200 + 150; // 350px
const finalWidthC = 100 + 75; // 175px

// 验证: 75 + 350 + 175 = 600px
```

场景二：空间不足 (flex-shrink)

设定:

- 容器宽度: 400px
- 子项 A: flex: 1 1 300px;
- 子项 B: flex: 1 2 200px;

```
.container {  
  width: 400px;  
  display: flex;  
}  
.a { flex: 1 1 300px; }  
.b { flex: 1 2 200px; }
```

1. 计算基准总宽

$$300px + 200px = 500px$$

2. 计算超出空间

$$500px - 400px = 100px \text{ (需要被压缩的总量)}$$

3. 计算收缩权重并分配

$$\text{A 的权重: } 300(\text{basis}) \times 1(\text{shrink}) = 300$$

$$\text{B 的权重: } 200(\text{basis}) \times 2(\text{shrink}) = 400$$

$$\text{总权重: } 300 + 400 = 700$$

$$\text{A 压缩: } 100 \times \frac{300}{700} \approx 42.86px$$

$$\text{B 压缩: } 100 \times \frac{400}{700} \approx 57.14px$$

4. 计算最终宽度

$$\text{A: } 300 - 42.86 \approx 257.14px$$

$$\text{B: } 200 - 57.14 \approx 142.86px$$

总结

Flexbox 的弹性伸缩并非魔法，而是一套定义明确的数学计算。

- 一切始于 `flex-basis`：它决定了是进入“扩张”模式还是“收缩”模式。
- 空间充足时，分配空间：系统根据 `flex-grow` 的比例分配剩余空间。
- 空间不足时，压缩空间：系统根据 `flex-basis` 和 `flex-shrink` 共同决定的权重分摊压缩量。